

Relational Databases and SQL II

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

SQL and NoSQL Landscape

The Evolution of Databases

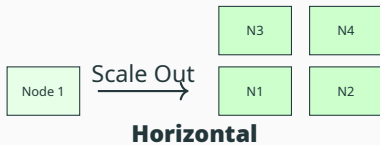
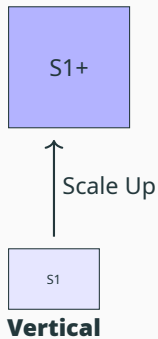
The database world has shifted from rigid tables to a diverse ecosystem.

- **1970s-80s:** RDBMS (Oracle, IBM) - Focus on storage efficiency.
- **1990s:** Object-Oriented DBs - Brief attempt at matching code and data.
- **2000s:** The Big Data Explosion - Google/Amazon needed massive scale.
- **2010s:** NoSQL Revolution - Focus on availability and flexible schemas.
- **Present:** Polyglot Persistence - Using the right tool for the right task.

Relational vs. Non-Relational

- **Relational (SQL):**
 - Predetermined schema (tables/rows/columns).
 - Strong consistency (ACID).
 - Best for structured data and complex relationships.
- **Non-Relational (NoSQL):**
 - Dynamic schema (documents, pairs, graphs).
 - High performance and easy horizontal scaling.
 - Best for rapid development and massive datasets.

Vertical vs. Horizontal Scaling



- **ACID (SQL Standard):**

- Atomicity, Consistency, Isolation, Durability.
- Data is always accurate and up-to-date across all clients.

- **BASE (NoSQL Standard):**

- **B**asically **A**vailable (The system responds even if nodes are down).
- **S**oft State (Data might change without input due to consistency rules).
- **E**ventual Consistency (Data will be consistent... eventually).

CAP Theorem

The CAP Theorem: Consistency

Consistency means that all clients see the same data at the same time, no matter which node they connect to.

- For this to happen, whenever data is written to one node, it must be instantly replicated to all other nodes.
- If a network partition occurs, the system must stop accepting writes to ensure consistency.

The CAP Theorem: Availability

Availability means that any client making a request for data gets a response, even if one or more nodes are down.

- In a distributed system, this is achieved through redundancy.
- However, if nodes cannot communicate (partition), they might return different (stale) data to ensure they always respond.

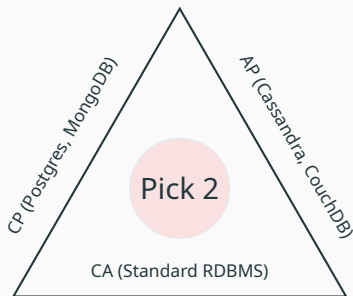
The CAP Theorem: Partition Tolerance

Partition Tolerance means that the system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes.

- In modern distributed systems, **Partition Tolerance is non-negotiable**. Network failures happen.
- Therefore, systems must choose between **Consistency (CP)** or **Availability (AP)** during a partition.

The CAP Triangle

Partition Tolerance (P)



Consistency (C)

Availability (A)

NoSQL Families

The Four NoSQL Families

To handle different data structures and scaling needs, four main families emerged:

- **Key-Value Stores:** Simple, fast, opaque data.
- **Document Stores:** Flexible, hierarchical, semi-structured data.
- **Wide-Column Stores:** Optimized for high-volume, sparse data.
- **Graph Databases:** Optimized for complex relationships and traversals.

Key-Value Stores

The simplest NoSQL model. Data is stored as an opaque value indexed by a unique key.

- **Analogy:** A Python Dictionary or a Java HashMap.
- **Strengths:** Extremely fast, great for simple data.
- **Usage:** Session management, user preferences, real-time leaderboards.
- **Example:** **Redis**, Amazon DynamoDB.

Redis Usage Example

```
# Basic operations
SET user:101 "Alice"
GET user:101           # Output: "Alice"

# Atomic increments
SET counter 10
INCR counter           # Output: 11

# Lists
LPUSH tasks "Email"
LPUSH tasks "Code"
RPOP tasks             # Output: "Email"
```

Stores data in JSON, BSON, or XML format. Documents are self-describing.

- **Analogy:** A folder of JSON files.
- **Strengths:** Flexible schema (different fields per document), intuitive for developers.
- **Usage:** Content management, e-commerce catalogs, user profiles.
- **Example: MongoDB, CouchDB.**

MongoDB Usage Example

```
// Inserting a document
db.users.insertOne({
  name: "Alice",
  age: 25,
  skills: ["SQL", "Python"]
});

// Querying with flexible filters
db.users.find({ age: { $gt: 20 } });

// Updating specific fields
db.users.updateOne(
  { name: "Alice" },
  { $set: { age: 26 } }
);
```

Wide-Column Stores

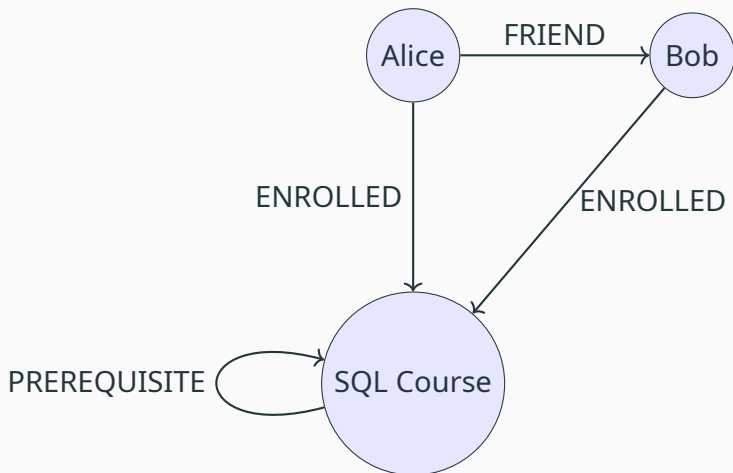
Stores data in column families instead of rows. Optimized for massive horizontal scale.

- **Analogy:** A 2D map where each row can have a different set of columns.
- **Strengths:** High write throughput, handles petabytes of data.
- **Usage:** Time-series data, historical logs, large-scale analytics.
- **Example:** **Apache Cassandra**, Google Bigtable.

Focuses on the relationships between entities. Data is represented as Nodes and Edges.

- **Analogy:** A social network or a map of flight routes.
- **Strengths:** Queries on complex relationships are much faster than SQL Joins.
- **Usage:** Recommendation engines, fraud detection, social networks.
- **Example:** **Neo4j**, Amazon Neptune.

Visualizing a Graph DB



Standalone SQL Databases

SQLite is a C-language library that implements a small, fast, self-contained, high-reliability, full-featured SQL database engine.

- **Embedded:** It is not a separate process. It is part of the application.
- **Zero-Config:** No setup or administration needed.
- **Atomic:** Transactions are fully ACID compliant even after crashes.
- **Cross-Platform:** The database file format is stable and portable.

- **Architecture:**

- Uses **B-Trees** for data storage on disk.
- Implements **WAL (Write-Ahead Log)** for improved concurrency.

- **When to use:**

- Local storage for mobile/desktop apps.
- Intermediate data during analysis.
- Low-to-medium traffic websites.

- **When to avoid:**

- High-concurrency write environments.
- Multi-server distributed applications.

H2 is a Java-based relational database that can be used in embedded or server mode.

- **Performance:** Extremely fast because it can run entirely in RAM.
- **Standard:** Supports standard SQL and JDBC.
- **Compatibility:** Can emulate the behavior of PostgreSQL, MariaDB, or Oracle.
- **Usage:** Unit testing, rapid prototyping, and caching layers in Java applications.

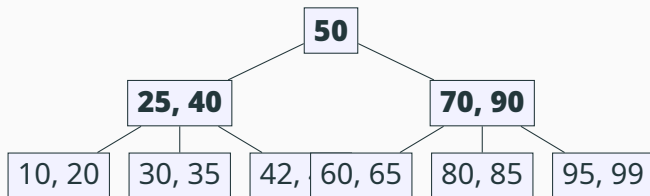
Advanced SQL: Performance

The Search Problem

Imagine a table `Students` with 1,000,000 records. How does the RDBMS find `name = 'Alice'`?

- **Full Table Scan ($O(N)$):**
 - The engine reads every single row from the first to the last.
 - If the record is at the end, it takes 1,000,000 operations.
- **Index Scan ($O(\log N)$):**
 - Using a sorted data structure (B-Tree).
 - For 1,000,000 records, it takes only ~ 20 operations.

B-Tree Visualization



The search starts at the root and follows pointers based on key ranges, drastically reducing the number of comparisons.

Index Management

```
-- Creating a standard index
CREATE INDEX idx_user_email ON users(email);

-- Composite index (Order matters!)
CREATE INDEX idx_name_age ON users(last_name, first_name);

-- Unique index (Enforces business logic)
CREATE UNIQUE INDEX idx_student_id ON students(id_number);

-- Removing an index
DROP INDEX idx_user_email;
```

Indexes are not “free.” They have a cost:

- **Write Penalty:** Every time you INSERT, UPDATE, or DELETE, the RDBMS must also update the index structure.
- **Disk Space:** Indexes take up significant space (sometimes as much as the data itself).
- **Maintenance:** Over time, indexes can become fragmented and require rebuilding (REINDEX or VACUUM).

How do you know if your index is actually being used?

-- SQLite

EXPLAIN QUERY PLAN SELECT * FROM users WHERE email = 'a@b.com';

-- Output: SEARCH TABLE users USING INDEX idx_user_email (email=?)

-- PostgreSQL / MySQL

EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'a@b.com';

- **Seq Scan:** Full table scan (Bad for large tables).
- **Index Scan:** Efficiently using the index (Good).

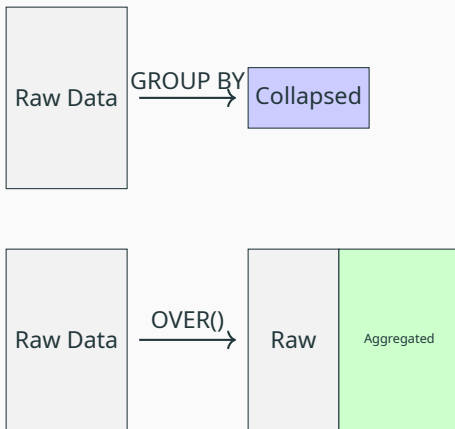
Advanced SQL: Analytics

Window Functions: Concept

Window functions perform a calculation across a set of table rows that are somehow related to the current row.

- **Key Difference:** Unlike GROUP BY, rows are not collapsed. Each row preserves its original identity.
- **Structure:** FUNCTION() OVER (PARTITION BY ... ORDER BY ...)
- **Functions:** SUM(), AVG(), RANK(), ROW_NUMBER(), LEAD(), LAG().

Visualizing Window Functions



Window Function Examples

```
-- Calculating a running total
```

```
SELECT date, amount,  
        SUM(amount) OVER (ORDER BY date) as running_total  
FROM sales;
```

```
-- Ranking items within a category
```

```
SELECT name, category, price,  
        RANK() OVER (PARTITION BY category ORDER BY price DESC) as pos  
FROM products;
```

```
-- Lag/Lead (Accessing previous/next row)
```

```
SELECT date, amount,  
        LAG(amount) OVER (ORDER BY date) as prev_day_amount  
FROM sales;
```

Common Table Expressions (CTE)

A CTE provides a way to define a temporary result set that you can reference within a single query.

- **Syntax:** WITH cte_name AS (SELECT ...)
SELECT ... FROM cte_name;
- **Benefits:**
 - **Readability:** Breaks complex queries into logical steps.
 - **Recursive Queries:** Allows querying trees or hierarchical data.
 - **Modular:** You can define multiple CTEs in one query.

CTE Example

```
WITH DeptAvg AS (  
    SELECT dept_id, AVG(salary) as avg_sal  
    FROM employees  
    GROUP BY dept_id  
)  
SELECT e.name, e.salary, d.avg_sal  
FROM employees e  
JOIN DeptAvg d ON e.id = d.dept_id -- Fix potential join id issue  
WHERE e.salary > d.avg_sal;
```

- Here, we first calculate the averages and then use that “virtual table” in the main query.

Recursive CTEs

Recursive CTEs are used to traverse hierarchical structures (like an organization chart).

```
WITH RECURSIVE subordinates AS (  
    -- Initial: Select the CEO  
    SELECT id, name, manager_id FROM employees WHERE name = 'CEO'  
    UNION ALL  
    -- Recursive: Join with subordinates  
    SELECT e.id, e.name, e.manager_id  
    FROM employees e  
    INNER JOIN subordinates s ON s.id = e.manager_id  
)  
SELECT * FROM subordinates;
```

Database Security

SQL Injection: The Attack

The most common database vulnerability. It occurs when user input is concatenated into a query string.

```
# VULNERABLE CODE
```

```
username = input("Enter name: ") # Input: ' OR '1'='1  
query = "SELECT * FROM users WHERE name = '" + username + "';"  
# Resulting SQL: SELECT * FROM users WHERE name = '' OR '1'='1';
```

- The attacker bypasses the logic and can extract, modify, or delete any data.

Prepared Statements: The Shield

Prepared statements (or Parameterized Queries) ensure that the RDBMS treats user input strictly as data, never as code.

```
# SECURE CODE
username = input("Enter name: ")
# 1. Template is sent to DB
# 2. Data is sent separately as a parameter
cursor.execute("SELECT * FROM users WHERE name = ?", (username,))
```

- **Workflow:**

1. **Prepare:** The DB parses, compiles, and optimizes the query plan.
2. **Execute:** The DB binds the values to the plan and runs it.

Prepared Statements: Efficiency

Beyond security, prepared statements offer performance benefits:

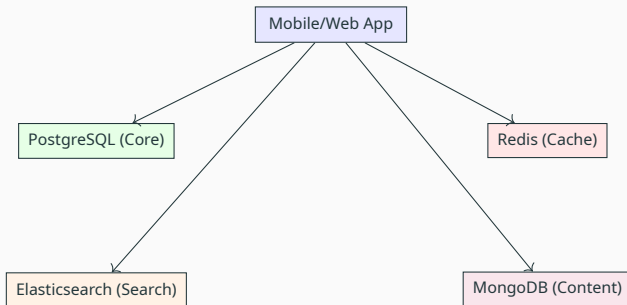
- **Binary Protocol:** Data is sent in a native binary format, reducing overhead.
- **Plan Reuse:** The RDBMS does not need to re-parse the query every time.
- **Safety:** Automatically handles character escaping and data types.

Modern Architectures

Modern applications rarely use just one database. They use the best tool for each sub-problem.

- **Orders (SQL):** Need ACID for financial transactions.
- **Product Catalog (NoSQL Document):** Need flexible schema for different products.
- **Product Search (Elasticsearch/Vector DB):** Need full-text or similarity search.
- **Session/Cache (Redis):** Need sub-millisecond latency.
- **Social Connections (Graph):** Need to find “Friends of Friends.”

The Modern Data Stack



Summary

Final Takeaways

- **Paradigm Shift:** SQL for structure and integrity; NoSQL for scale and flexibility.
- **Standalone:** SQLite and H2 are essential tools for many development scenarios.
- **Performance:** Indexing is the single most important factor for read speed.
- **Analytics:** Window Functions and CTEs are standard tools for modern data analysis.
- **Security:** Prepared statements are mandatory for any production application.
- **Complexity:** Modern systems are hybrid (Polyglot Persistence).